

PyTorch Training Workflow

What is Artificial Neural Network (ANN)?

Machine Learning (ML) is a branch of artificial intelligence that gives computers **the ability to learn from data and improve at a task without being explicitly programmed for every rule**. Instead of writing fixed instructions like:

```
if email contains "free money" then mark as spam
```

ML algorithms find patterns automatically from examples. You give the system many examples of inputs and expected outputs, and it learns the mapping between them. Common types of ML:

- **Supervised learning:** learns from labeled data, e.g. spam vs. not spam.
- **Unsupervised learning:** finds patterns in unlabeled data, e.g. grouping similar customers.
- **Reinforcement learning:** learns by trial and error with rewards, e.g. game-playing AI.

What is an ANN?

An Artificial Neural Network (ANN) is a machine learning model inspired by the structure of the human brain. It is made of many connected units called neurons or nodes, organized in layers. ANNs are especially good at finding complex patterns in data, such as images, text, audio, and many other kinds of signals. A typical ANN has:

- **Input layer:** receives the raw data.
- **Hidden layers:** one or more layers that learn intermediate representations.
- **Output layer:** produces the final result, like a class label or a number.

When a network has many hidden layers, it is often called a deep neural network, and training it is called deep learning.

How an ANN Works

1. **Neurons and connections** Each neuron receives inputs from previous neurons. Every connection has a weight, which controls how important that input is. A single neuron does three things:
 - Multiply each input by its weight.
 - Add all those products together plus a bias.
 - Pass the result through an activation function.

Mathematically:

```
output = activation( sum( input_i * weight_i ) + bias )
```

2. Activation functions

The activation function introduces non-linearity, allowing the network to learn complex patterns.

Common activation functions:

ReLU – returns 0 for negative values, otherwise the value itself.

Sigmoid – squashes values between 0 and 1.

Tanh – squashes values between -1 and 1.

Without activation functions, the whole network would just be a linear model, no matter how many layers it has.

3. Forward propagation

When data is passed through the network, it goes layer by layer from input to output. This is called forward propagation.

Example:

Input: pixels of an image of a cat.

Hidden layers: detect edges, then shapes, then parts like ears and eyes.

Output layer: probability that the image is a cat, dog, etc.

4. Loss function

After the output is produced, the network compares it to the correct answer using a loss function or cost function. This gives a single number representing how wrong the network is.

Common loss functions:

Mean squared error – for regression.

Cross-entropy – for classification.

5. Backpropagation

To improve, the network needs to know how much each weight contributed to the error. Backpropagation calculates the gradient of the loss with respect to every weight by moving backward from the output layer to the input layer.

It uses the chain rule from calculus to efficiently compute these gradients.

6. Updating weights

The weights are then adjusted in the direction that reduces the loss. This is done using an optimization algorithm, usually gradient descent or a variant like Adam.

Update rule:

$\text{weight} = \text{weight} - \text{learning_rate} * \text{gradient}$

The learning rate controls how big the update steps are.

7. Training loop

The whole process repeats many times over the entire dataset:

Forward pass – make a prediction.

Compute loss – measure error.

Backward pass – compute gradients.

Update weights – improve the model.

One full pass over the training data is called an epoch. Training often requires many epochs.

8. Inference

Once training is complete, the network is used for inference or prediction on new, unseen data. At this stage, only forward propagation happens; no learning occurs.

Quick Reference: The Six Steps

In order to understand how to train do machine learning with PyTorch, we shall use a simple example of recognising handwritten digits (MNIST dataset). The following six steps outline the core workflow,

from loading data to saving a trained model. Each step is accompanied by a brief description of its physical intuition.

Step	Core Function
Load Data	<code>datasets.MNIST()</code> <code>DataLoader</code>
Build the Model Architecture	<code>nn.Conv2d</code> <code>nn.Linear</code> <code>nn.ReLU</code> <code>nn.MaxPool2d</code> <code>nn.Flatten</code>
Choose loss function and optimizer	<code>nn.CrossEntropyLoss</code> <code>optim.Adam</code>
Training	<code>model()</code> <code>loss.backward()</code> <code>optimizer.step()</code>
Save and reload model weights	<code>model.state_dict()</code> <code>torch.save()</code> <code>torch.load()</code>
GPU acceleration	<code>.to("cuda")</code>

Building the Network Architecture

The following two code snippets implement the architecture described in Step 2. Both are valid `nn.Module` subclasses, accept grayscale images of size (1, 28, 28), and output **10 raw logits** – no softmax or sigmoid at the final layer, because `CrossEntropyLoss` internally applies a softmax.

Plain ANN (Multilayer Perceptron)

Intuition: Flatten the 28×28 image into a 784-dimensional vector and pass it through a few dense layers. This ignores spatial relationships and treats each pixel as an independent feature.

```
import torch
import torch.nn as nn

class ThreeLayerMLP(nn.Module):
    def __init__(self):
        # The general structure goes like this:
        # L1 -> ReLU -> L2 -> ReLU -> L3 -> Sigmoid
        self.fc1 = nn.Linear(784, 128) # First Layer
        self.fc2 = nn.Linear(128, 64) # Second Layer
        self.fc3 = nn.Linear(64, 10) # Third Layer

    def forward(self, x):
        # This is how the data flows through the network
        # x is the input tensor of shape (batch_size, 784)
        x = nn.ReLU(self.fc1(x)) # First Layer + ReLU
        x = nn.ReLU(self.fc2(x)) # Second Layer + ReLU
        x = nn.Sigmoid(self.fc3(x)) # Third Layer + Sigmoid
        return x # Output tensor of shape (batch_size, 10)
```

Listing 1: Plain ANN implementation (MLP)

2.2 Convolutional Neural Network (LeNet-style CNN)

Intuition: Preserve the spatial structure. Use sliding filters (convolutions) to extract local features (edges, corners), then pool to reduce dimensions. Finally, flatten the condensed feature maps and feed them into dense layers.

```
class HybridMLP(nn.Module):
    def __init__(self):
        # Kernel size is 3x3, meaning that we take a 3x3 patch
        #   of the image and apply the convolution operation to it.
        # The stride is 1, meaning that we move the kernel by 1 pixel at a time.
        # The padding is 0, meaning that we do not add any extra pixels
        #   around the image.
        # Each out channels corresponds to a different filter
        #   that is applied to the image, and the number of out channels
        #   is 32, meaning that we have 32 different filters
        #   that are applied to the image.
        # In channels is 1, meaning that the input image has 1
        #   channel (grayscale image).
        self.conv1 = nn.Conv2d(
            in_channels=1, out_channels=32, kernel_size=3
        ) # First Convolutional Layer
        # Pooling layer is used to REDUCE the spatial dimensions of the image,
        # and to make the network more robust to small translations in the image.
        self.pool = nn.MaxPool2d(kernel_size=2)
        self.conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3)

        self.fc1 = nn.Linear(64 * 5 * 5, 64) # First Fully Connected Layer
        self.fc2 = nn.Linear(64, 10) # Second Fully Connected Layer

    def forward(self, x):
        # This is how the data flows through the network
        x = nn.ReLU(self.conv1(x)) # First Convolutional Layer + ReLU
        x = self.pool(x) # First Pooling Layer
        x = nn.ReLU(self.conv2(x)) # Second Convolutional Layer + ReLU
        x = self.pool(x) # Second Pooling Layer

        x = x.view(x.size(0), -1) # Flatten the tensor for the fully
        #   connected layers

        x = nn.ReLU(self.fc1(x)) # First Fully Connected Layer + ReLU
        x = nn.Sigmoid(self.fc2(x)) # Second Fully Connected Layer + Sigmoid
        return x # Output tensor of shape (batch_size, 10)
```

Listing 2: LeNet-style CNN implementation

3. Comparison at a Glance

Aspect	Plain ANN	CNN
Input handling	Directly flattens to 784 – loses spatial layout	Keeps 3D structure (channels, height, width)
Feature extraction	Dense layers connect all pixels globally	Local receptive fields via convolution (weight sharing)
Parameter count	Larger (100k for 784→128)	Much smaller (e.g., $3 \times 3 \times 32 = 288$ weights)
Best for	Data where pixel positions are irrelevant (e.g., tabular)	Images, speech, or any data with local correlations
Forward flow	Flatten → Linear → ReLU → Linear → ... → 10	Conv → ReLU → Pool → Conv → ReLU → Pool → Flatten → Linear → ... → 10

Important: Both examples do not apply Softmax or Sigmoid at the final layer. This is because Step 3 uses `nn.CrossEntropyLoss`, which internally applies a softmax to the logits. Adding an extra activation would interfere with the loss calculation.